

2025/2026

Project Winter

Γραφικά και Εικονική Πραγματικότητα (ECE AK709)



ΓΕΡΟΝΤΑΣ ΝΙΚΟΛΑΟΣ

A.M.: 1092813

Πίνακας περιεχομένων

Εισαγωγή.....	2
Part A.....	2
Ερώτημα 1.....	2
Ερώτημα 2.....	5
Ερώτημα 3.....	6
Ερώτημα 4.....	7
Part B.....	8
Ερώτημα 5.....	8
Ερώτημα 6.....	10
Ερώτημα 7.....	11
Υποερώτημα α	11
Υποερώτημα β	11
Υποερώτημα γ.....	12
Υποερώτημα δ	13
Bonus	15
Δομή Project	19
Bloopers	20
Βιβλιογραφία	21

Εισαγωγή

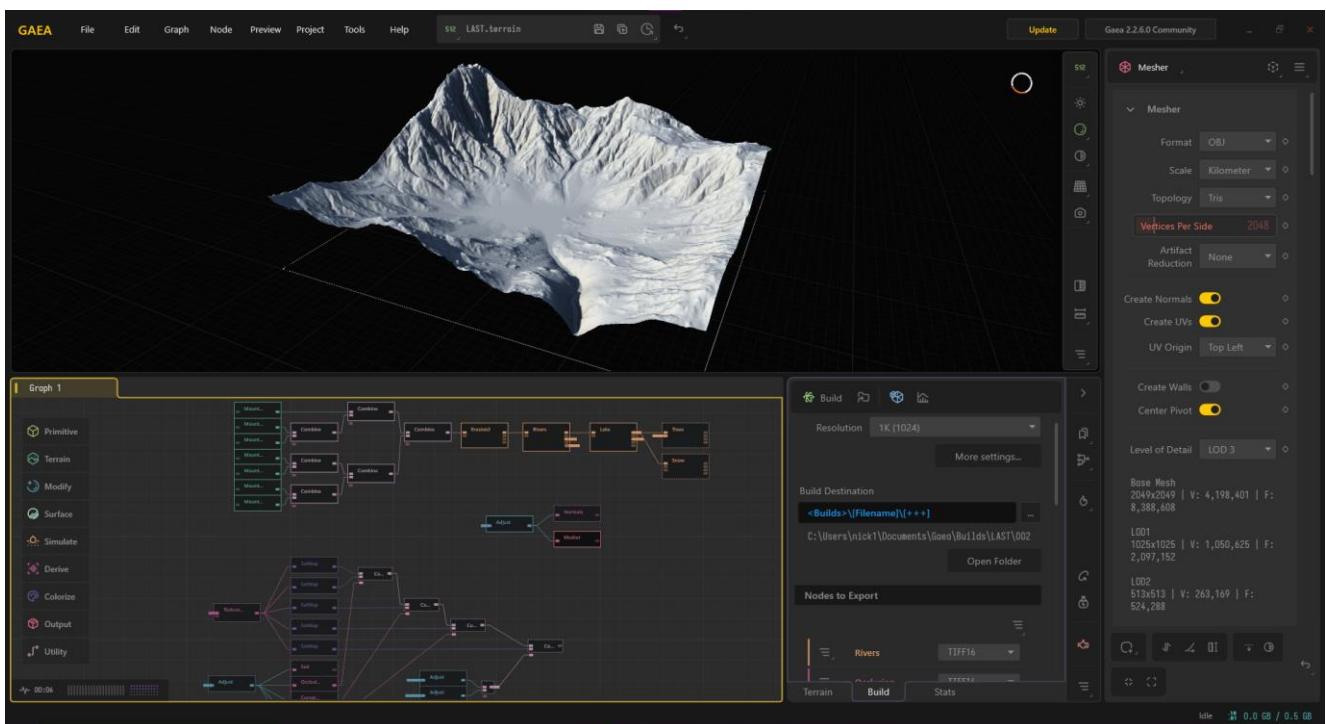
Η παρούσα εργασία αφορά τη δημιουργία ενός δυναμικού 3D περιβάλλοντος με θέμα τον χειμώνα, υλοποιημένου σε C++ με χρήση OpenGL και GLSL. Στόχος ήταν η ανάπτυξη ενός ρεαλιστικού τοπίου που συνδυάζει τη στατική διαμόρφωση του χώρου με τη δυναμική εξέλιξή του μέσω της χιονόπτωσης, του παγώματος της λίμνης και της συσσώρευσης χιονιού. Ακολουθεί η αναλυτική παρουσίαση της υλοποίησης των ζητούμενων, όπως ορίζονται στην εκφώνηση.

Part A

Ερώτημα 1

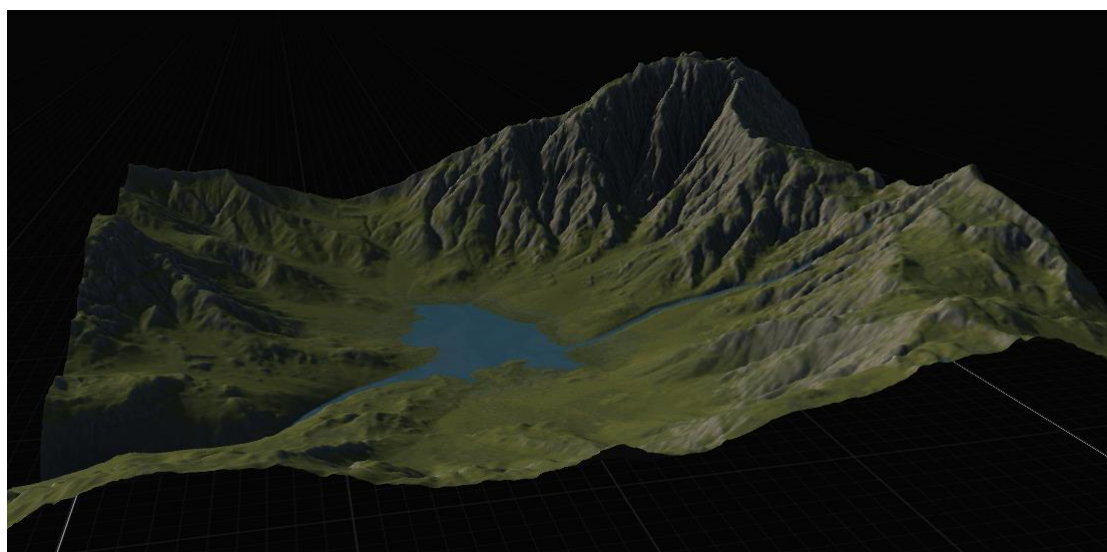
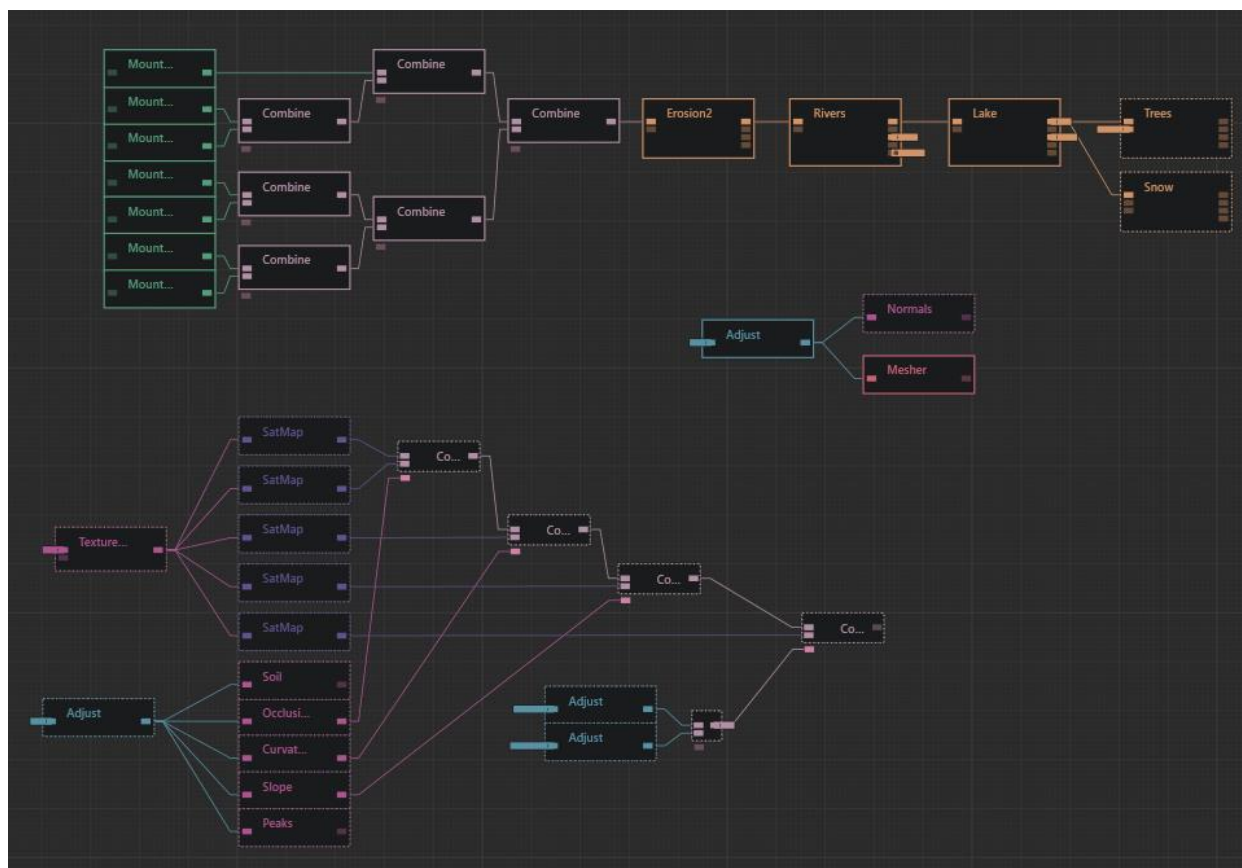
Load a mountainous terrain (or generate one, if you prefer). Make sure there is a valley amidst the mountains. Add clouds in the sky and make them look as realistic as you can.

Για τη δημιουργία του ορεινού terrain με ενσωματωμένη κοιλάδα χρησιμοποιήθηκε το λογισμικό Gaea (Community Edition) της QuadSpinner, το οποίο επιτρέπει διαδικαστική παραγωγή ρεαλιστικών γεωμορφολογιών μέσω erosion και river simulation. Η κοιλάδα προέκυψε φυσικά από τη ροή των ποταμών και τη διάβρωση του εδάφους.



Σημειώνεται ότι το Gaea έχει αξιοποιηθεί και σε τίτλους όπως Alan Wake II, Death Stranding 2 και God of War Ragnarök.

Στο παρακάτω σχήμα φαίνεται ο γράφος ροής που χρησιμοποιήθηκε για τη διαδικαστική παραγωγή του terrain.



Παράλληλα, για την ρεαλιστική εμφάνιση των νεφών, χρησιμοποιήθηκε η τεχνική των instanced billboard particles, όπου κάθε σύννεφο αποτελείται από ένα cluster 50 οντοτήτων. Η φυσική τους μορφή προκύπτει από το blending 2 διαφορετικών noise textures (cloudBase και cloudDetail), ενώ η χρήση της συνάρτησης smoothstep στον fragment shader εξασφαλίζει απαλά άκρα, αποφεύγοντας τις απότομες γραμμές διαχωρισμού.

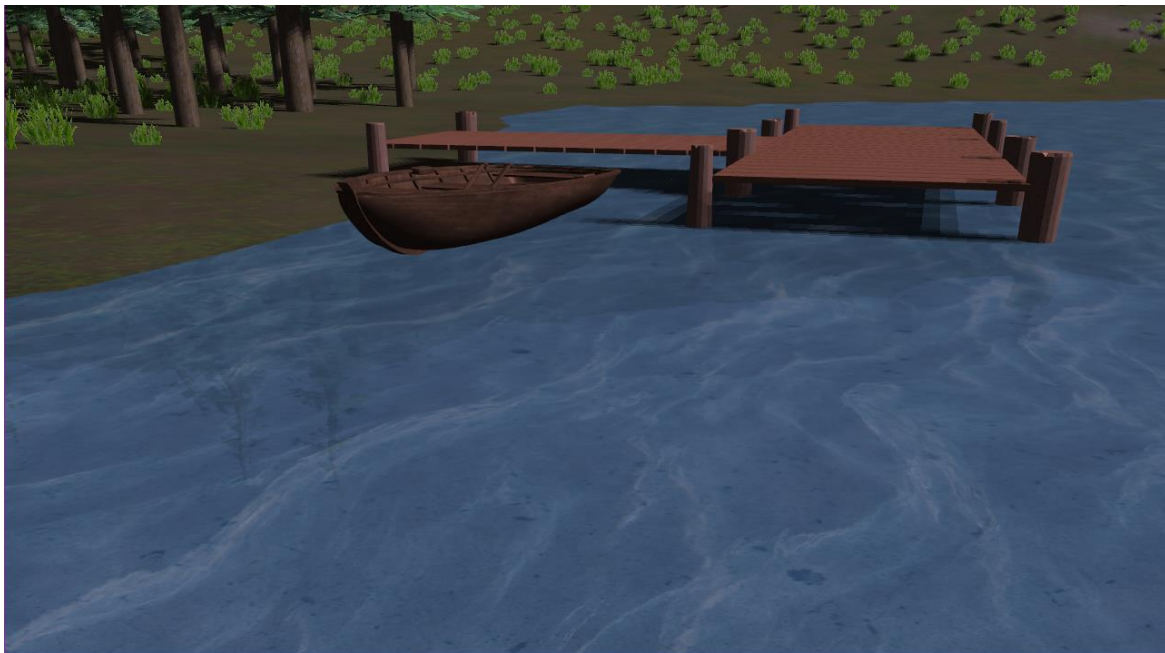
Η αίσθηση του τρισδιάστατου όγκου προσομοιώνεται μέσω vertical gradient φωτισμού, ο οποίος βάφει το κάτω μέρος του σύννεφου με σκιές και την κορυφή με λευκό φως. Τέλος, εφαρμόζεται δυναμική κίνηση με looping logic και dynamic billboarding, χρησιμοποιώντας τα Right και Up vectors της κάμερας, ώστε τα σύννεφα να είναι πάντα στραμμένα προς τον παίχτη, ενισχύοντας την αίσθηση του βάθους και τον ρεαλισμό.



Ερώτημα 2

Inside the valley, create a small lake. Make the water look realistic by using multiple water textures with displacement (like we did in lab 3).

Η λίμνη δημιουργήθηκε στο [Ερώτημα 1](#) με την βοήθεια του Gaea. Το νερό/υφή της υλοποιήθηκε στον ShadowMapping.fragmentshader και συνδυάζει πολλαπλά textures. Συγκεκριμένα, χρησιμοποιήθηκαν τεχνικές texture scrolling και displacement mapping, όπου οι συντεταγμένες UV του νερού παραμορφώνονται δυναμικά βάσει του χρόνου και τριγωνομετρικών συναρτήσεων για τη δημιουργία κυματισμών. Επιπλέον, ενσωματώθηκαν αντανακλάσεις του περιβάλλοντος σε μεταγενέστερο στάδιο, καθώς ούτως ή άλλως απαιτούνταν για την παγωμένη λίμνη, οι οποίες υφίστανται την ίδια παραμόρφωση με την επιφάνεια του νερού για πιο ρεαλιστικό αποτέλεσμα .



Ερώτημα 3

Add a directional light source to act as the sun. Make sure to adjust the appropriate light properties to make the light fit in a winter setting (e.g., blueish color tones, low intensity). Also add shadows using any technique you want.

Η υλοποίηση του φωτισμού και της σκίασης βασίζεται στο μοντέλο Phong και στην τεχνική Shadow Mapping. Λόγω του χειμερινού τοπίου, η πηγή φωτός έχει ψυχρούς τόνους ($L_a = L_d = L_s = [0.8, 0.8, 1.0]$). Μια κρίσιμη βελτίωση είναι η τεχνική Frustum Fitting, όπου το οπτικό πεδίο της πηγής προσαρμόζεται δυναμικά στο frustum της κάμερας, μεγιστοποιώντας την ανάλυση του shadow map στην ορατή περιοχή. Παράλληλα, για επιπλέον βελτιστοποίηση της απόδοσης, εφαρμόζεται φίλτρο PCF με 3×3 kernel μόνο για τις κοντινές σκιές (απόσταση < 80), ενώ για τις απομακρυσμένες σκιές χρησιμοποιείται απλό 1-tap sampling. Το τελικό αποτέλεσμα ενισχύεται από την ομίχλη, η οποία επιτρέπει τον περιορισμό του far plane.



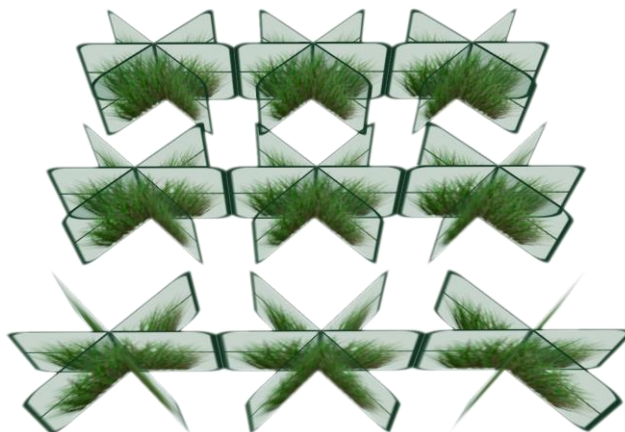
Ερώτημα 4

Add sparse vegetation (patches of grass, short bushes, etc.).

Για την προσθήκη βλάστησης (γρασίδι και απομακρυσμένα δέντρα) στο περιβάλλον, χρησιμοποιήθηκε το Python script `grass_positions_generation.py` για την δημιουργία των θέσεων των billboard clusters και η κλάση `Meadow` σε επίπεδο C++.

Συγκεκριμένα, το Python script αναλαμβάνει την τοποθέτηση των clusters βάση υψομέτρου και των masks `Lake` και `River`. Στα masks εφαρμόζεται η μέθοδος `binary dilation` για να υπάρξει κενό μεταξύ βλάστησης και νερού.

Στην C++ πλευρά, για το γρασίδι δημιουργούνται 3 επίπεδα quads τοποθετημένα σε γωνία 60 μοιρών μεταξύ τους, ενώ για τα μακρινά δέντρα χρησιμοποιούνται 2 επίπεδα. Έτσι, δίνεται η ψευδαίσθηση του τρισδιάστατου όγκου από οποιαδήποτε γωνία θέασης. Δεδομένου ότι τα αρχεία `.bmp` δεν υποστηρίζουν κανάλι άλφα (διαφάνεια), χρησιμοποιείται η επιλογή `useTransparentTex` στους fragment shaders φωτισμού και σκίασης, η οποία απορρίπτει pixels συγκεκριμένης απόχρωσης που θεωρούνται διαφανή (για τα assets μου). Τέλος, μέσω της παραμέτρου `windPower`, δίνεται δυναμική κίνηση στη βλάστηση, προσομοιώνοντας την ταλάντωση λόγω του ανέμου ([Ερώτημα 6](#)).



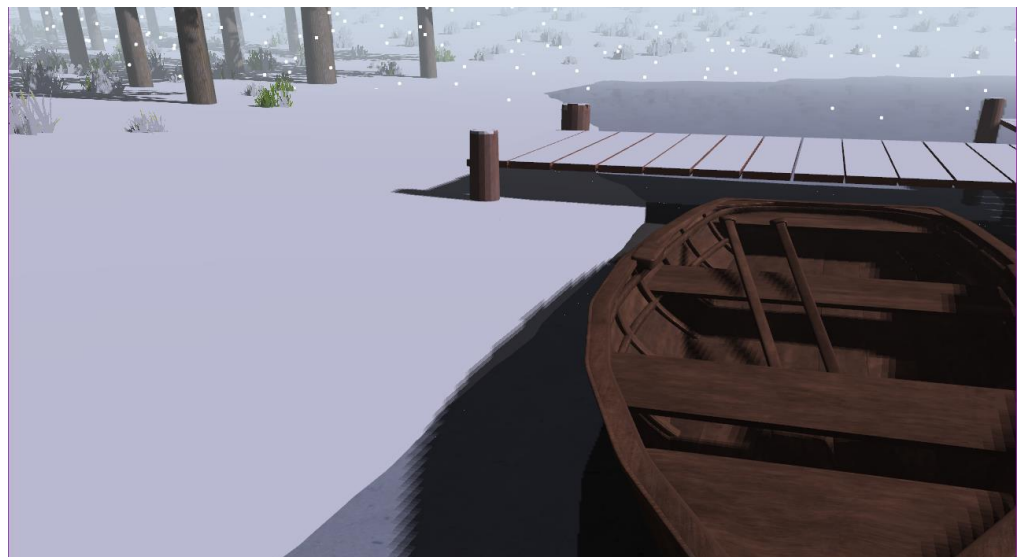
Part B

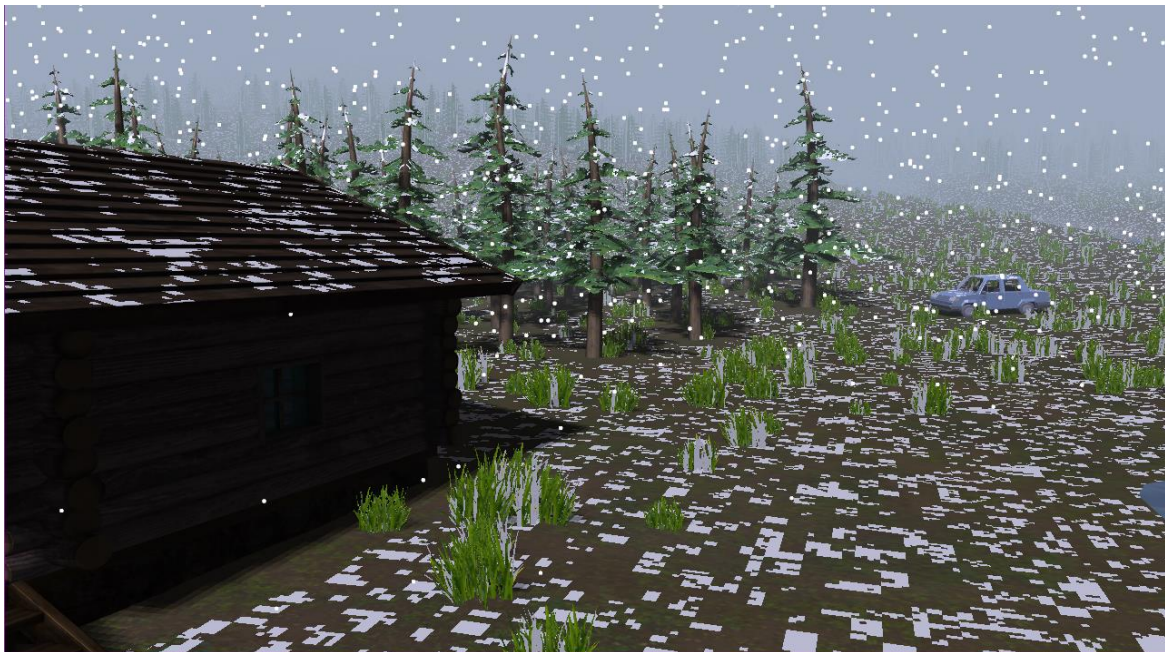
Ερώτημα 5

Create the effect that it's snowing. The snowfall is initially disabled and starts with the press of a key on the keyboard. While the snowfall continues, snow builds up on top of the terrain and the landscape becomes "snowy". The lake freezes and the water turns into ice. The environment is reflected atop the ice lake's surface.

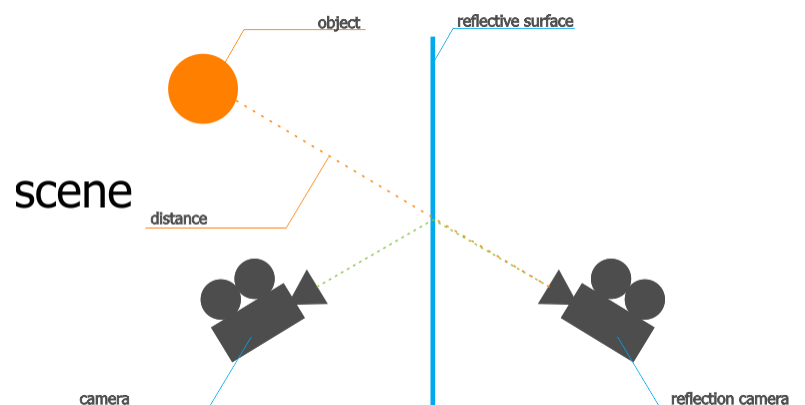
Η υλοποίηση της χιονόπτωσης και της συσσώρευσης χιονιού βασίζεται στο συνδυασμό σωματιδίων και δυναμικών shaders. Συγκεκριμένα, χρησιμοποιήθηκε ένα particle system 4000 σωματιδίων, τα οποία κινούνται σε ένα «κουτί» γύρω από την κάμερα (view space) και ανακυκλώνονται (wrap-around logic) για την ψευδαίσθηση ασταμάτητης πτώσης.

Παράλληλα, για τη ρεαλιστική προσθήκη χιονιού στο περιβάλλον, χρησιμοποιήθηκε η κλάση SnowSource η οποία, μέσω της τεχνικής Shadow Mapping, υπολογίζει ποιες επιφάνειες είναι εκτεθειμένες στον ουρανό ώστε να «χιονιστούν», αποφεύγοντας περιοχές κάτω από υπόστεγα ή δέντρα. Η συσσώρευση γίνεται σταδιακά μέσω της παραμέτρου snowAmount με τη χρήση noise στον fragment shader για την απόδοση μεμονωμένων νιφάδων, ενώ ο όγκος του χιονιού προσομοιώνεται στον vertex shader με την τεχνική Taubin Smoothing (Snow Inflation), η οποία μετατοπίζει τα vertices κατά μήκος των normals τους.





Τέλος, το πάγωμα της λίμνης επιτυγχάνεται με τον μηδενισμό της κίνησης του νερού ($\text{effectiveTime} = 0$) και την τροποποίηση του texture της. Οι αντανάκλασεις στη λίμνη γίνονται πιο έντονες και υλοποιούνται με την τεχνική του Planar Reflection μέσω ενός ξεχωριστού FBO. Η διαδικασία γίνεται με τον υπολογισμό ενός mirrored view matrix, ο οποίος προκύπτει από τον πολλαπλασιασμό ενός πίνακα αντανάκλασης με τον τρέχοντα πίνακα view της κάμερας.



Κατά το `reflection_pass`, η σκηνή γίνεται render σε ένα texture, ενώ παράλληλα εφαρμόζεται ένα clipping plane στη στάθμη της λίμνης (`WATER_HEIGHT`) για να απορριφθούν αντικείμενα που βρίσκονται κάτω από αυτήν και αντιστρέφεται το culling (`glCullFace(GL_FRONT)`) λόγω της αλλαγής της φοράς των τριγώνων.



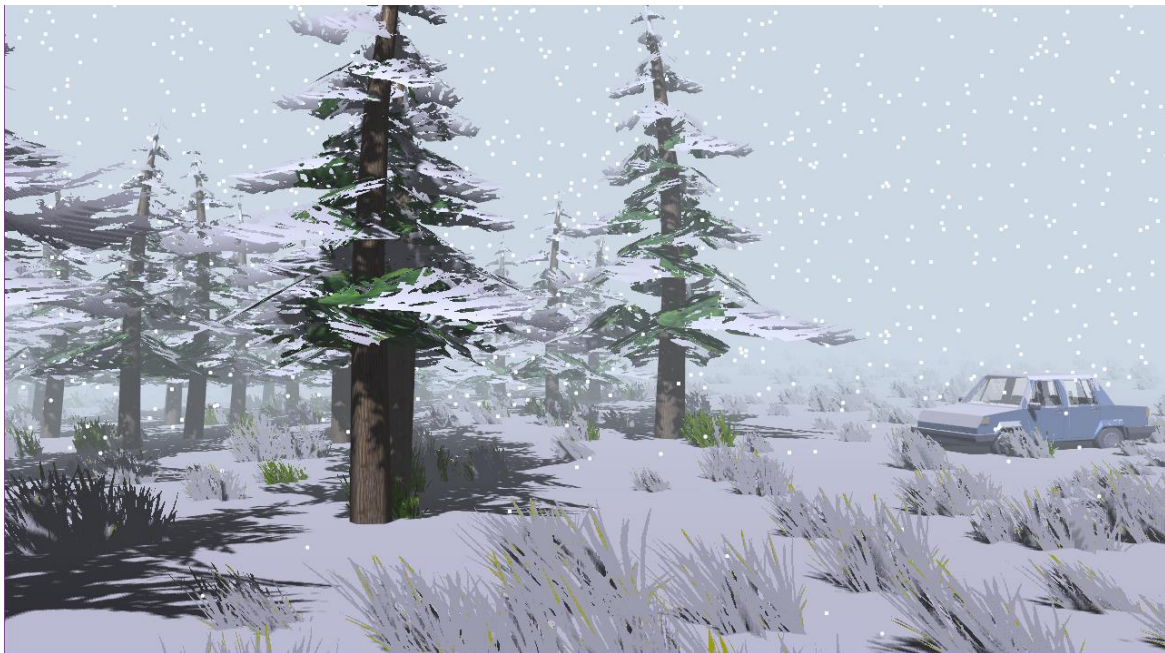
Επιπλέον, για ακόμα πιο ρεαλιστικό αποτέλεσμα, ενσωματώθηκε *exponential fog*, η οποία υπολογίζεται δυναμικά στον fragment shader βάσει της απόστασης των αντικειμένων από την κάμερα.

Ερώτημα 6

By pressing a second key, strong wind starts blowing, affecting the snowfall and vegetation movement.

Η υλοποίηση του αέρα βασίζεται στη τεχνική procedural vertex displacement, όπου η κίνηση της βλάστησης υπολογίζεται εξ' ολοκλήρου στον vertex shader. Συγκεκριμένα, χρησιμοποιείται η γ συντεταγμένη των UVs ως συντελεστής stiffness (0 στη ρίζα, 1 στην κορυφή), επιτρέποντας στα δέντρα και το γρασίδι να ταλαντώνονται οργανικά βάσει τριγωνομετρικών συναρτήσεων που συνδυάζουν τον χρόνο και την θέση τους, ώστε τα γειτονικά αντικείμενα να μην κινούνται πανομοιότυπα. Η ένταση της ταλάντωσης κλιμακώνεται δυναμικά μέσω της παραμέτρου windPower.

Παράλληλα, ο αέρας επιδρά και στο particle system της χιονόπτωσης προσθέτοντας μια οριζόντια δύναμη (windVelocity) στην ταχύτητα κάθε νιφάδας (C++ υλοποίηση).



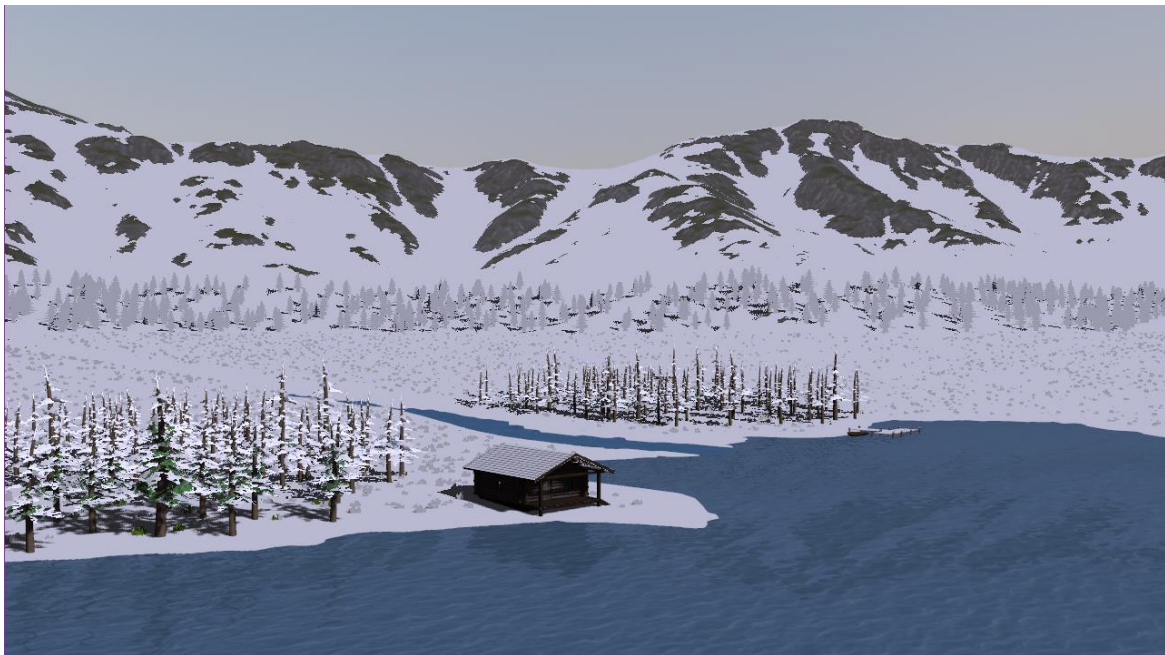
Ερώτημα 7

Create the following effects:

Υποερώτημα α

- With the press of a third key, the clouds start to clear away and the sun comes out. Adjust the light properties accordingly.

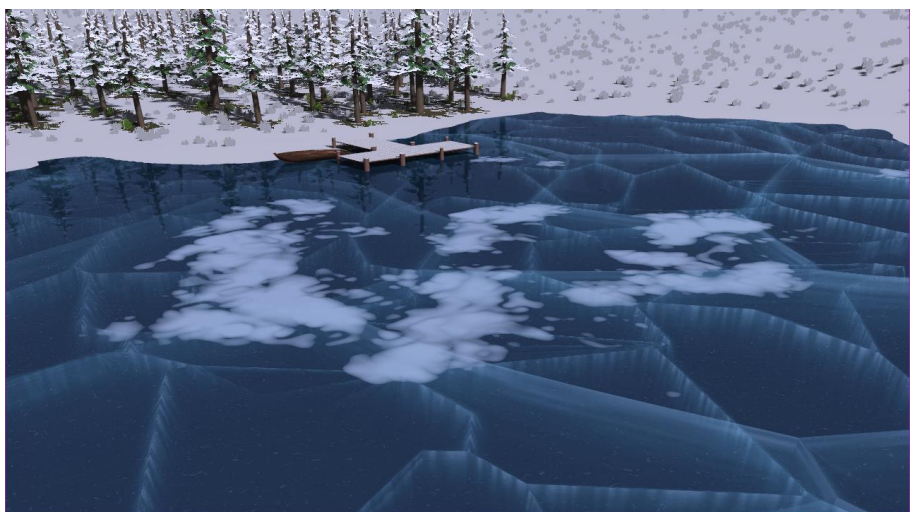
Με το πάτημα του πλήκτρου C, ενεργοποιείται η κατάσταση `forceClearFog`. Η μετάβαση δεν είναι ακαριαία, αλλά επιτυγχάνεται μέσω της βαθμιαίας μείωσης της παραμέτρου `fogDensity`, η οποία υποχωρεί με σταθερό ρυθμό μέχρι να μηδενιστεί, αποκαλύπτοντας σταδιακά το χιονισμένο τοπίο. Ταυτόχρονα, η χιονόπτωση τερματίζεται και απενεργοποιείται η σχεδίαση του `cloudSystem`.



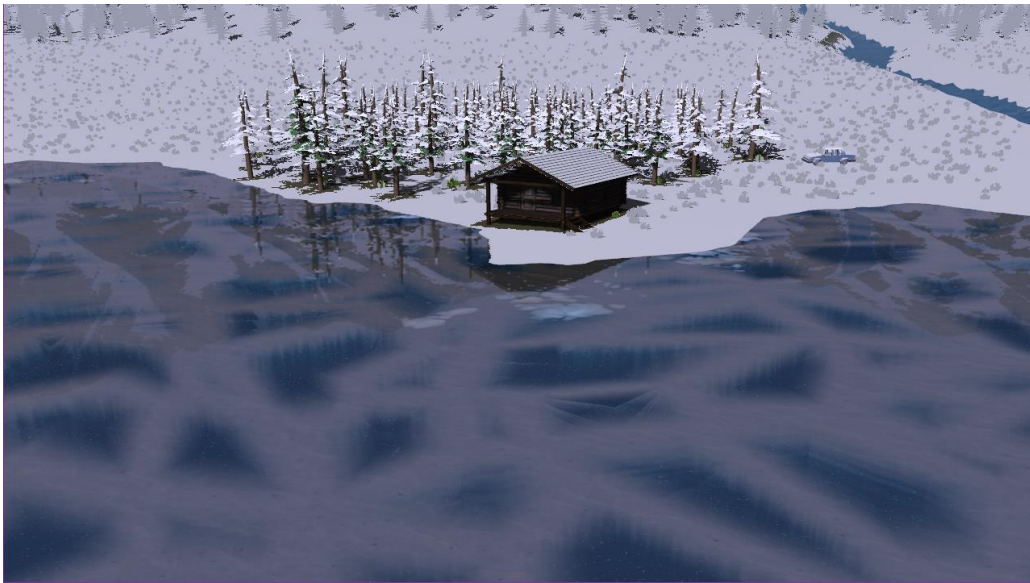
Υποερώτημα β

- The ice in the lake appears to crack and melts back into water.

Η δημιουργία των ρωγμών στην παγωμένη λίμνη υλοποιήθηκε με τον αλγόριθμο "Frozen Lake" του Alexander Alekseen (TDM), ο οποίος παράγει procedural μοτίβα μέσω Voronoi noise στον fragment shader. Η μετάβαση από τη στερεή στην υγρή μορφή πραγματοποιείται σταδιακά καθώς μειώνεται το `fogDensity`, με το λιώσιμο να ξεκινά από το εσωτερικό των ρωγμών προς τα έξω. Για τον σκοπό αυτό, χρησιμοποιούνται συναρτήσεις απόστασης (distance fields) από τις γραμμές των cracks (`mapCracks1`, `mapCracks2`), ώστε η παράμετρος `meltCoverage` να προσομοιώνει την φυσική εξάπλωση του νερού. Στην τελική κατάσταση, η μεταβλητή `effectiveTime` παύει να είναι μηδενική



και λαμβάνει ξανά την τιμή του χρόνου. Αυτό έχει ως αποτέλεσμα να ενεργοποιηθούν ξανά το texture scrolling και το displacement mapping, επαναφέροντας τη λίμνη στην αρχική της μορφή, όπως αυτή ορίστηκε στο [Ερώτημα 2](#).

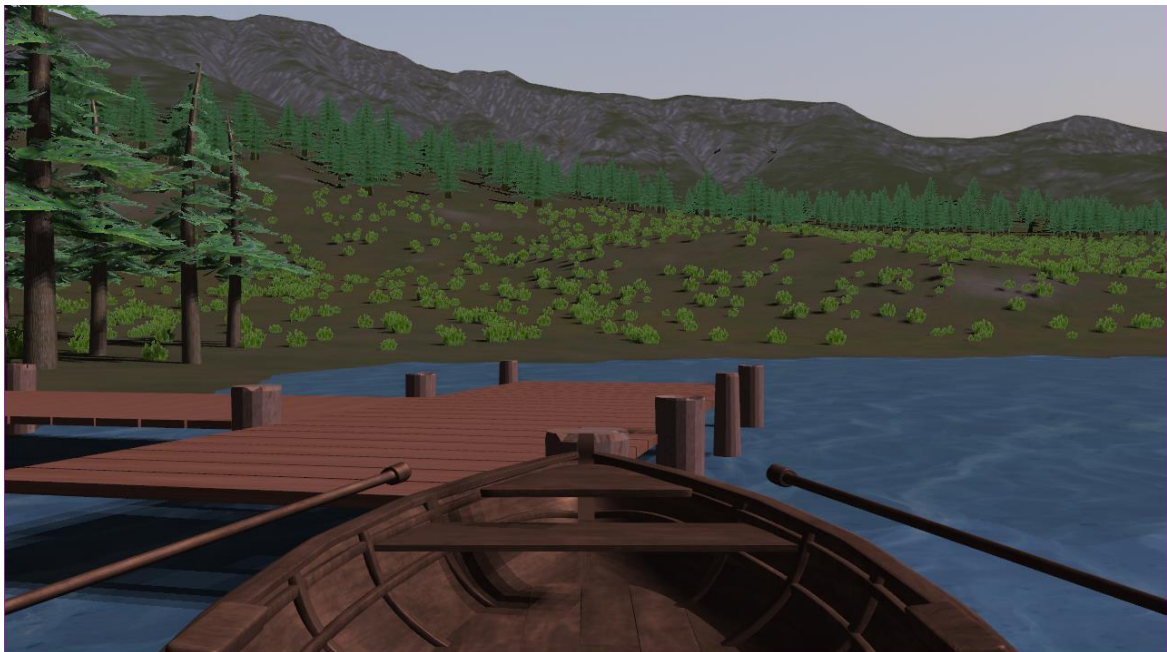


Υποερώτημα γ

- The snow that was collected in the environment begins melting and starts to roll downhill, filling the lake further.

Το συγκεκριμένο υποερώτημα δεν υλοποιήθηκε και τη θέση του πήρε η δυνατότητα χειρισμού μιας βάρκας στη λίμνη. Όταν ο παίκτης βρίσκεται εντός της βάρκας, εκτελείται η εντολή `glClear(GL_DEPTH_BUFFER_BIT)` ακριβώς πριν το rendering του μοντέλου, ώστε η βάρκα να σχεδιάζεται πάντα πάνω (μπροστά) και να μην «κόβεται». Η κίνηση της βάρκας προσομοιώνεται με αδράνεια, ενώ έχει προστεθεί ένα procedural wobble μέσω τριγωνομετρικών συναρτήσεων για την ρεαλιστική αναπαράσταση της πλεύσης σε κύματα. Τέλος, τα κουπιά διαθέτουν δυναμικό animation συγχρονισμένο με ηχητικά εφέ, τα οποία παίζουν αυτόματα κάθε φορά που το μοντέλο του κουπιού φτάνει στο κατώτατο σημείο της τροχιάς του μέσα στο νερό.

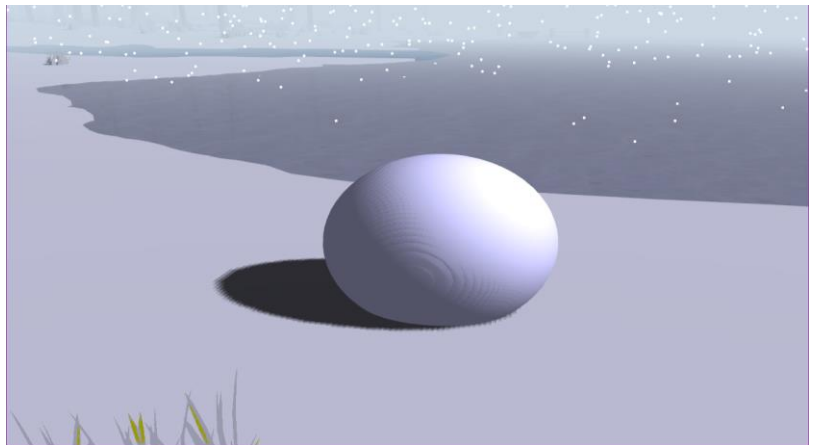


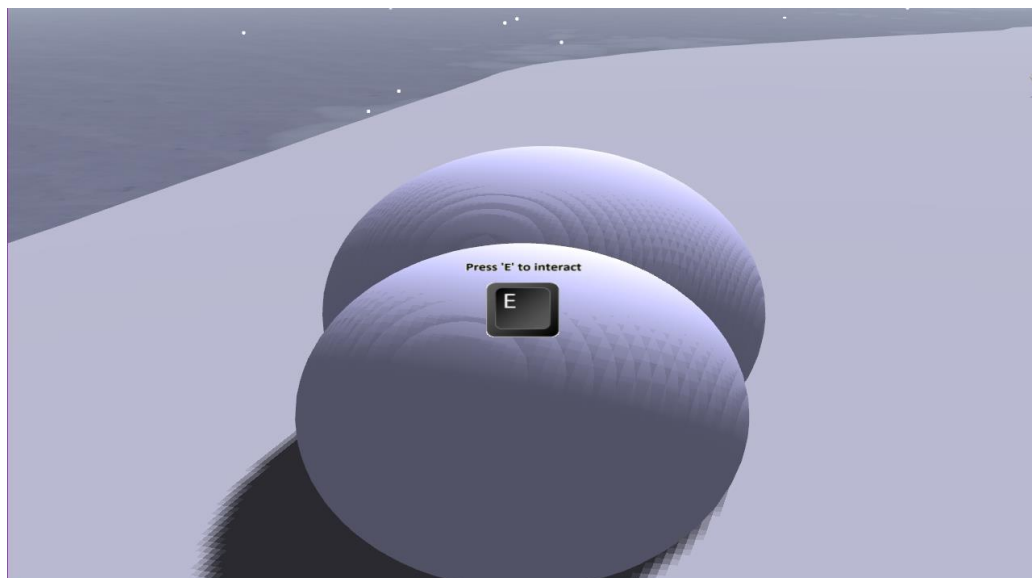


Υποερώτημα δ

- As the melting of the snow accelerates, an avalanche is caused.

Το υποερώτημα της χιονοστιβάδας δεν υλοποιήθηκε και τη θέση του πήρε ένα διαδραστικό σύστημα δημιουργίας χιονάνθρωπου. Η υλοποίηση στηρίζεται στις κλάσεις Snowball και Snowman, όπου ο παίκτης μπορεί να δημιουργήσει μια αρχική μπάλα χιονιού (εφόσον το snowAmount > 0.8f) και να την κυλήσει στο έδαφος. Κατά την κύλιση, η μπάλα αυξάνει δυναμικά την ακτίνα και τη μάζα της μέσω ενός growth rate όσο κινείται, προσομοιώνοντας τη συσσώρευση χιονιού, ενώ η κίνησή της περιορίζεται/γίνεται στο το ύψος του terrain μέσω της συνάρτησης getTerrainHeight. Μετά τη δημιουργία 2 σφαιρών, ο παίκτης μπορεί να τις στοιβάξει. Η τελική «μεταμόρφωση» σε μοντέλο χιονάνθρωπου γίνεται μέσω ενός Frustum Visibility check: το σύστημα ελέγχει αν οι στοιβαγμένες μπάλες βρίσκονται εντός των planes του frustum της κάμερας και, τη στιγμή που ο παίκτης στρέφει το βλέμμα του αλλού, τις αντικαθιστά αυτόματα με το τελικό μοντέλο.

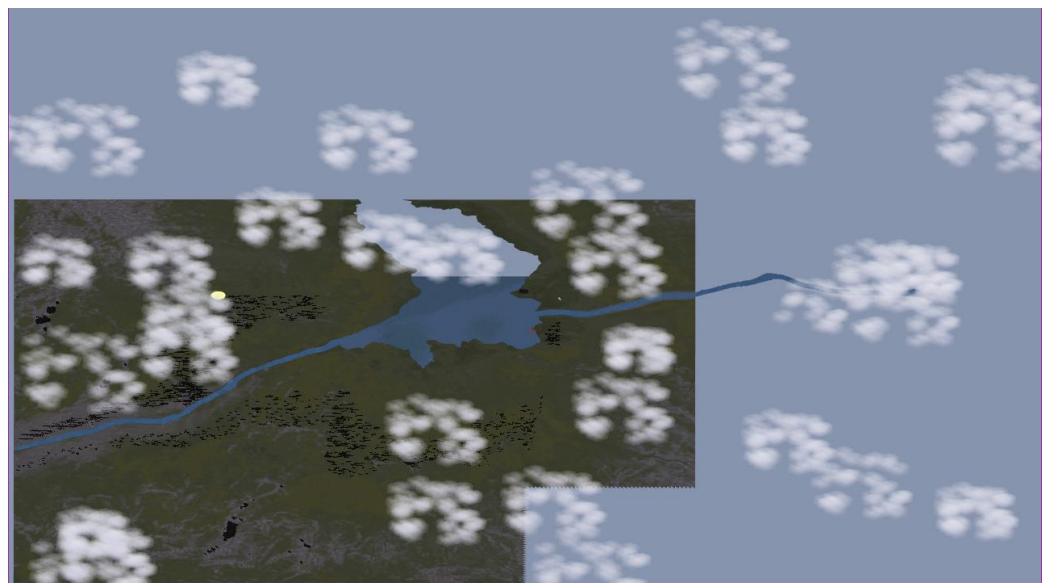
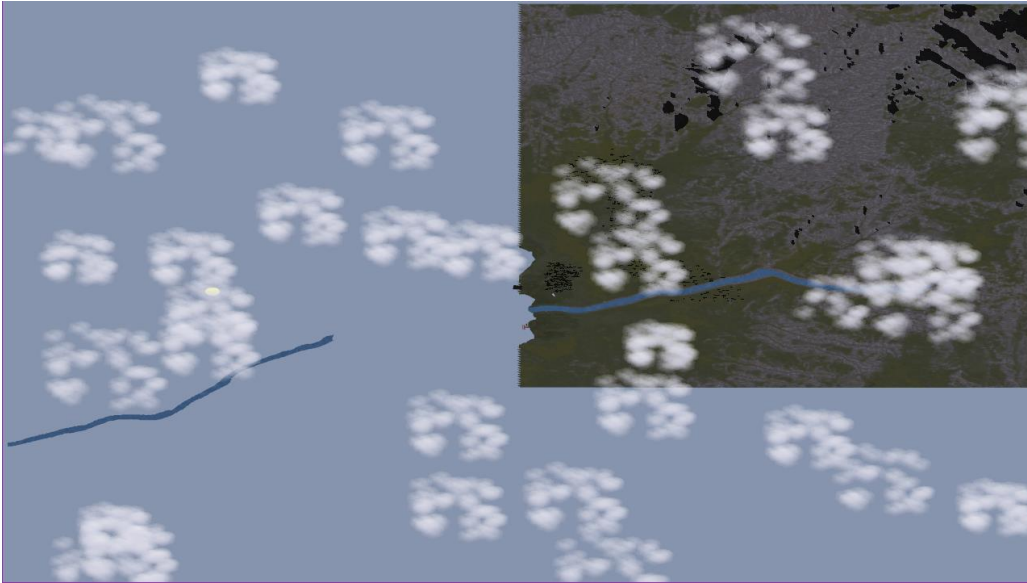




Bonus

Βελτιστοποίηση Terrain και Collision System

Η βελτιστοποίηση της απόδοσης του terrain επιτυγχάνεται μέσω της τεχνικής View Frustum Culling, όπου το περιβάλλον χωρίζεται σε chunks (land και lake) και σχεδιάζονται μόνο όσα βρίσκονται εντός του «κώνου» της κάμερας, βάσει ελέγχων AABB (isBoxInFrustum) έναντι των επιπέδων του frustum. Ο όγκος του παίκτη προσομοιώνεται με τη χρήση μιας σφαίρας (PLAYER_RADIUS), ενώ η ανίχνευση συγκρούσεων πραγματοποιείται σε τοπικό επίπεδο για κάθε Drawable αντικείμενο, συνδυάζοντας γρήγορους ελέγχους AABB και ακριβή έλεγχο τριγώνων. Ειδικοί αόρατοι τοίχοι σύγκρουσης (lakeWall, worldWall) εξασφαλίζουν τα όρια του κόσμου και εμποδίζουν την είσοδο στη λίμνη όταν αυτή δεν είναι παγωμένη.



Δυναμικά Animations και Κατοπτρισμοί

Οι πόρτες της καλύβας και του αυτοκινήτου διαθέτουν δυναμικά animations που βασίζονται στο interpolation μεταξύ τρέχουσας και επιθυμητής γωνίας περιστροφής γύρω από έναν άξονα (hingePosition). Ιδιαίτερη έμφαση δόθηκε στον καθρέπτη του αυτοκινήτου, ο οποίος υλοποιεί έναν 2^ο κύκλο σχεδίασης (mirror pass). Οι παράμετροι του καθρέπτη (θέση και normal) ενημερώνονται δυναμικά σε σχέση με την κίνηση της πόρτας, εξασφαλίζοντας ότι η αντανάκλαση παραμένει γεωμετρικά συνεπής ακόμη και κατά τη διάρκεια του animation.



Αλληλεπίδραση Ray Marching και Ήχος

Η αλληλεπίδραση με τα αντικείμενα της σκηνής (πόρτα, arcade) βασίζεται στην τεχνική του Ray Marching, όπου πραγματοποιείται δειγματοληψία κατά μήκος της κατεύθυνσης της κάμερας για τον εντοπισμό σημείων ενδιαφέροντος σε συγκεκριμένη απόσταση. Το ηχητικό σύστημα είναι βασισμένο στη βιβλιοθήκη miniaudio. Οι ήχοι των βημάτων μεταβάλλονται αυτόματα ανάλογα με τον τύπο του εδάφους (χώμα, ξύλο, πάγος, χιόνι), ο οποίος ανιχνεύεται μέσω της θέσης του παίκτη και των αντίστοιχων terrain masks ή της κατάστασης της λίμνης.



Ενσωμάτωση Arcade (Java) και Ending Sequence

Το σύστημα Arcade ενσωματώνει μια εξωτερική εφαρμογή Java εντός του 3D περιβάλλοντος . Η εφαρμογή OpenGL εκκινεί το Java project και, μέσω του Windows API (BitBlt, GetDIBits), παίρνει τα pixels του παραθύρου σε πραγματικό χρόνο, ενημερώνοντας ένα texture που προβάλλεται σε ένα 3D quad εντός της καλύβας . Η ίδια λογική χρησιμοποιήθηκε και για την τελική σκηνή, όπου ένα Python script αναλαμβάνει την αναπαραγωγή βίντεο μέσω OpenCV, ενώ μια αυτοματοποιημένη ακολουθία εντολών (dogSequence) προσομοιώνει την αλληλεπίδραση του σκύλου με το περιβάλλον του παιχνιδιού.



Διαχείριση Μενού και Χρόνου (ImGui)

Η υλοποίηση του pause menu πραγματοποιήθηκε με τη χρήση της βιβλιοθήκης ImGui μέσω της κλάσης MenuGUI. Μια κρίσιμη λεπτομέρεια είναι ο διαχωρισμός μεταξύ πραγματικού και simulated time: όταν το μενού ενεργοποιείται, ο πραγματικός χρόνος συνεχίζει να τρέχει, αλλά η μεταβλητή simulatedDeltaTime μηδενίζεται.



ElevenLabs

Για την αφήγησης, ενσωματώθηκαν voiceovers τα οποία παράχθηκαν με τη χρήση τεχνητής νοημοσύνης μέσω της πλατφόρμας ElevenLabs. Επεξεργάστηκαν με την βοήθεια του Audacity.

Προσαρμογή Ύψους

Η κίνηση του παίκτη και των αντικειμένων (οι χιονόμπαλες συγκεκριμένα) στον χώρο γίνεται από ένα σύστημα δυναμικής προσαρμογής ύψους που βασίζεται στη γεωμετρία του εδάφους. Μέσω της συνάρτησης getTerrainHeight, το σύστημα εκτελεί γραμμική παρεμβολή σε βαρυκεντρικές συντεταγμένες πάνω στα τρίγωνα του terrain, υπολογίζοντας το ακριβές υψόμετρο για οποιαδήποτε (x, z) θέση.

Η χρήση της Python υπήρξε καθοριστική για την αυτοματοποίηση της προετοιμασίας των assets και τη διαχείριση της γεωμετρίας του κόσμου.

Δομή Project

```
.
└─ ProjectWinter/
    ├── PW_report.pdf
    ├── Winter.pdf
    └─ SourceCode/
        ├── CMakeLists.txt
        ├── common/
        ├── external/
        ├── FAILS/
        ├── PYTHON_HELPERS/
        └─ project_winter/
            ├── main.cpp
            ├── assets/
            ├── shaders/
            └─ src/
```

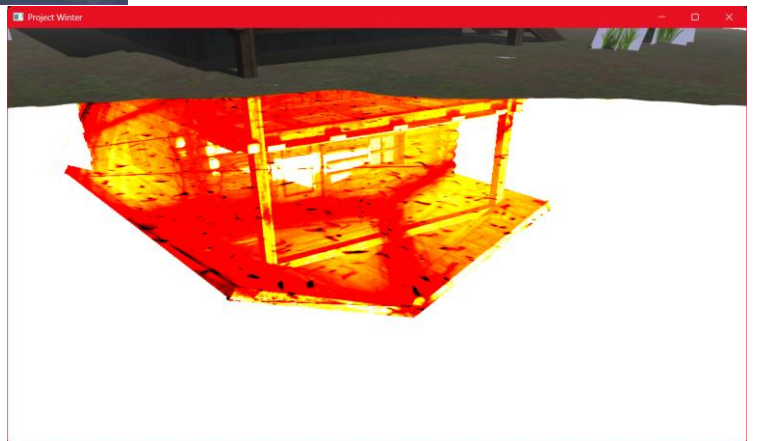
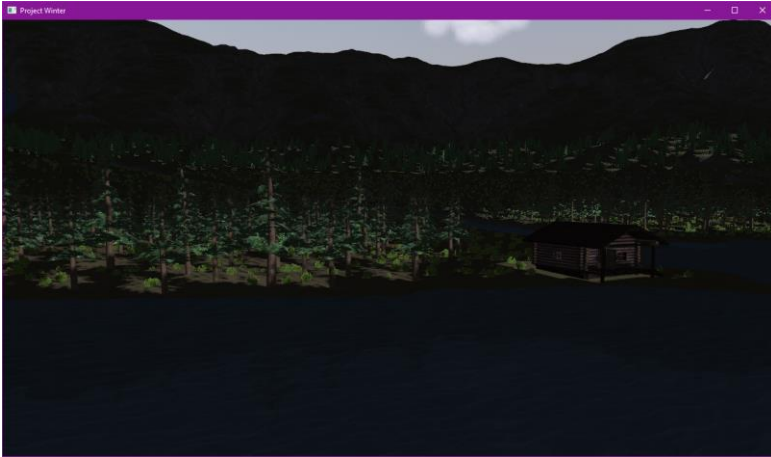
Το project είναι οργανωμένο έτσι ώστε να διαχωρίζονται τα γραφικά (C++/OpenGL) από τα εργαλεία αυτοματοποίησης και τα assets:

- project_winter/: Ο κύριος κώδικας (main.cpp, src/) και οι shaders.
- common/ & external/: Βοηθητικές βιβλιοθήκες.
- PYTHON_HELPERS/: Scripts για mesh manipulation και video playback.
- FAILS/: Bloopers

Οδηγίες Setup & Εκτέλεσης

- ✂ Βασικό Project: Αναλυτικές οδηγίες για το build και την εκκίνηση υπάρχουν μέσα σε κάθε Lab directory του GitHub μου.
- ✂ Arcade Game (Java): Οι οδηγίες και ο κώδικας για το παιχνίδι "The Forbidden Spaceship" βρίσκονται στο αντίστοιχο [GitHub Repository](#).
- ✂ Python (Video Ending): Απαιτείται η εγκατάσταση της βιβλιοθήκης opencv-python στην κεντρική έκδοση Python του συστήματος.
- ✂ Game Assets: Λόγω μεγέθους, τα απαραίτητα 3D models και textures βρίσκονται στον εξής σύνδεσμο: [Google Drive](#).

Bloopers



Βιβλιογραφία

1. <https://quadspinner.com/>
2. <https://3dworldgen.blogspot.com/2015/08/volumetric-clouds.html>
3. <https://iquilezles.org/articles/dynclouds/>
4. <https://vulpinii.github.io/tutorials/grass-modelisation/en/>
5. <https://learn.microsoft.com/en-us/windows/win32/dxtecharts/common-techniques-to-improve-shadow-depth-maps#calculating-a-tight-projection>
6. <https://www.meshlab.net/#download>
7. <https://www.gdcvault.com/play/1024103/Technical-Art-Techniques-of-Naughty>
8. <https://cgv.cs.nthu.edu.tw/download/file?guid=2707896d-4da2-11e6-a355-00113247b9b1>
9. <https://www.shadertoy.com/view/MsXyzN>
10. https://www.reddit.com/r/cpp/comments/159aln9/why_is_imgui_so_highly_liked/
11. <https://github.com/ocornut/imgui/tree/master>
12. <https://learnopengl.com/Advanced-OpenGL/Instancing>
13. https://github.com/nothings/stb/blob/master/stb_vorbis.c
14. <https://miniaud.io/>
15. <https://github.com/leadedge/SpoutProcessing/tree/master>
16. <https://developer.unigine.com/en/docs/future/objects/lights/planar/>

Assets:

- a. https://polyhaven.com/a/coast_sand_rocks_02
- b. <https://opengameart.org/textures/5566+5571+5571+5593+5593+5595+5595+5567+5567+5568+5568+5569+5569+5594+5594+5570+5570>
- c. https://www.turbosquid.com/3d-models/wooden-rowboat-2295812?dd_referrer=
- d. <https://sketchfab.com/3d-models/scandinavian-log-cabin-e89718f746484189902162a91ae0b504>
- e. <https://pixabay.com/sound-effects/nature-grass-field-walking-sound-354514/>
- f. https://polyhaven.com/a/qwantani_moonrise
- g. <https://opengameart.org/content/library-of-game-sounds>